Here's an example of reimplementing a while loop using a method. Because the condition is passed as a block, it can be evaluated each time around the loop:

```
def my_while(cond, &body)
  while cond.call
    body.call
  end
end

a = 0
my_while(-> { a < 3 }) do
  puts a
  a += 1
end
```

produces:

```
0
1
2
```

Block Parameter Lists

When you are using the -> syntax, you declare the parameters in a separate list before the block body, similar to a method definition. Blocks written using the other syntax forms declare their parameter lists between vertical bars. In both cases, the parameter list looks just like the list you can give to methods. It can take default values, splat arguments (described later on page 89), keyword arguments, and its own block parameter (a trailing argument starting with an ampersand). You can write blocks that are just as versatile as methods. Actually, they are more versatile, because these blocks are also closures, while methods are not. Here's a block using the lambda notation:

```
proc1 = lambda do |a, *b, &block|
  puts "a = #{a.inspect}"
  puts "b = #{b.inspect}"
  block.call
end

proc1.call(1, 2, 3, 4) { puts "in block1" }
```

produces:

```
a = 1
b = [2, 3, 4]
in block1
```

And here's one using the -> notation:

```
proc2 = -> (a, *b, &block) do
  puts "a = #{a.inspect}"
  puts "b = #{b.inspect}"
  block.call
end

proc2.call(1, 2, 3, 4) { puts "in block2" }
```

produces:

```
a = 1
b = [2, 3, 4]
in block2
```